

LABORATÓRIO TPSE I



Prática 05: Implementando Interrupção para GPIO e Timers

Prof. Francisco Helder

10 de junho de 2026

Sumário

1	Funcionalidade Interrupts	2
1.1	O Controlador de Interrupções (MPU_INTC/AINTC)	2
1.2	Tipos de Interrupção: IRQ e FIQ	2
1.3	Interrupt Vector Table	3
2	Configurando Interrupção para GPIO	6
2.1	Rotina de Serviço de Interrupção - ISR	7
3	Configurando Interrupção para Timer	8
3.1	Função Delay com Interrupção	9
3.2	Rotina de Serviço de Interrupção - ISR	9
3.3	Circuito Completo	10
3.4	Circuito Completo	10
4	Atividades Práticas	12

1 Funcionalidade Interrupts

Uma interrupção pode ser usada para sinalizar ao processador todos os tipos de eventos - por exemplo, porque os dados chegaram e podem ser lidos, um usuário acionou uma chave ou um determinado período de tempo se passou.

As interrupções permitem que os desenvolvedores separem as operações de tempo crítico do programa principal para garantir que sejam processadas de maneira priorizada. Como as interrupções são eventos assíncronos, elas podem ocorrer a qualquer momento durante a execução do programa principal.

Logo, uma interrupção é o mecanismo que o processador usa para pausar o que está fazendo e atender um objetivo paralelo. Esse objetivo pode ser algo como “oh, ei, alguém apertou o botão. Vamos ver o que é”. Ou a busca paralela pode ser a troca de contexto entre diferentes threads. Existem 128 eventos de interrupção exclusivos que podem ocorrer na Beaglebone Black.

Vamos usar interrupções para receber a ação de clicar no botão e contagem cronológica, e assim realizar alteração na lógica do seu programa.

1.1 O Controlador de Interrupções (MPU_INTC/AINTC)

O módulo responsável pelo gerenciamento de interrupções é chamado de MPU_INTC (também referenciado como AINTC — ARM Interrupt Controller). Ele é mapeado em memória no endereço 0x48200000 e ocupa um espaço de 0x1000 bytes. MIT

O INTC atua como intermediário entre os periféricos do SoC e o núcleo ARM, filtrando, priorizando e encaminhando os eventos de interrupção.

1.2 Tipos de Interrupção: IRQ e FIQ

O ARM Cortex-A8 suporta dois tipos fundamentais de interrupção:

- IRQ (Interrupt Request): interrupção de propósito geral, usada pela maioria dos periféricos.
- FIQ (Fast Interrupt Request): interrupção de alta prioridade, com latência menor.

O FIQ é a interrupção de mais alta prioridade e pode preemptar tanto os handlers de IRQ quanto seções do kernel onde as IRQs estão desabilitadas. Além disso, o handler FIQ não pode ser preemptado por outras IRQs, garantindo latências previsíveis e determinísticas. STMicroelectronics Cada uma das 128 linhas de interrupção pode ser configurada individualmente como IRQ ou FIQ.

As prioridades são configuradas através dos registradores INTC_ILR_x (onde x vai de 0 a 127), descritos no TRM do AM335x na Seção 6.5.1.44. Cada registrador ILR define a prioridade daquela linha de interrupção específica. O INTC suporta 64 níveis de prioridade (0 = maior prioridade). Quando múltiplas interrupções estão pendentes, o hardware seleciona automaticamente a de maior prioridade para ser servida primeiro.

Também existe o mecanismo de Priority Masking (limiar de prioridade): é possível adicionar um threshold para acelerar ainda mais o atendimento de interrupções de alta prioridade, conforme descrito na Seção 6.1.1.2.2 do TRM

Preempção de Interrupções (Nested Interrupts)

O AM335x suporta interrupções aninhadas (nested): uma interrupção de maior prioridade pode preemptar o handler de uma interrupção de menor prioridade que já esteja em execução. O TRM documenta:

- INTC Processing Sequence – fluxo normal de atendimento de interrupção.
- INTC Preemptive Processing Sequence – fluxo com preempção entre interrupções.
- Spurious Interrupt Handling – tratamento de interrupções espúrias (falsas).

Todos esses fluxos estão documentados no TRM do AM335x (SPRUH73H), na seção de Interrupt Controller Registers, com os registradores INTC detalhados a partir da Seção 6.5. Principais Registradores são mostrado na Tabela 1:

Tabela 1: Registradores base para interrupção no AM335x.

Registrador	Função
INTCPSz_SYSCONFIG	Configuração geral do INTC
INTCPS_SIR_IRQ	Identifica a IRQ ativa de maior prioridade
INTCPS_SIR_FIQ	Identifica a FIQ ativa de maior prioridade
INTCPS_CONTROL	Sinaliza fim de interrupção (New IRQ/FIQ Agree)
INTCPS_MIR_n	Máscara de interrupção (habilita/desabilita por grupo)
INTCPS_ILR_m	Configura prioridade e tipo (IRQ/FIQ) de cada linha
INTCPS_PENDING_IRQ_n	Indica quais IRQs estão pendentes

1.3 Interrupt Vector Table

Uma tabela de vetores de interrupção é uma lista de instruções e geralmente fica como as primeiras linhas de código em seu aplicativo. O processador é codificado para chamar a enésima instrução para um evento de interrupção específico. Aqui está um exemplo de tabela de vetores de interrupção (*Interrupt Vector Table - IVT*) em assembly:

```

1 .equ INTC_BASE, 0x48200000
2
3 /* CPSR */
4 .equ CPSR_I, 0x80
5 .equ CPSR_F, 0x40
6 .equ CPSR_IRQ, 0x12
7 .equ CPSR_USR, 0x10
8 .equ CPSR_FIQ, 0x11
9 .equ CPSR_SVC, 0x13
10 .equ CPSR_ABT, 0x17
11 .equ CPSR_UND, 0x1B
12 .equ CPSR_SYS, 0x1F
13
14 /* Vector table */
15 _vector_table:
16     ldr    pc, _reset          /* reset - _start          */
17     ldr    pc, _undf           /* undefined - _undf       */
18     ldr    pc, _swi            /* SWI - _swi              */
19     ldr    pc, _pabt           /* program abort - _pabt   */

```

```

20  ldr    pc, _dabt      /* data abort - _dabt      */
21  nop                      /* reserved      */
22  ldr    pc, _irq      /* IRQ - read the VIC      */
23  ldr    pc, _fiq      /* FIQ - _fiq      */
24
25 _reset: .word _start
26 _undf:  .word 0x4030CE24 /* undefined      */
27 _swi:   .word 0x4030CE28 /* SWI            */
28 _pabt:  .word 0x4030CE2C /* program abort   */
29 _dabt:  .word 0x4030CE30 /* data abort      */
30      nop
31 _irq:   .word 0x4030CE38 /* IRQ            */
32 _fiq:   .word 0x4030CE3C /* FIQ            */

```

Dado o **IVT**, você pode implementar essas diferentes funções para lidar com cada operação de interrupção respectiva.

Para configurarmos nossa própria tabela de vetores de interrupção, nós copiaremos o endereço da função que queremos invocar para um ponto específico da memória. Essa abstração significa que seu código pode ser executado em qualquer lugar e você ainda poderá configurar o **IVT**. De acordo com a seção 26.1.4.2 do manual de referência técnica, podemos armazenar o endereço de nosso manipulador **Handler** nos locais apresentados na Figura 1:

Address	Exception	Content
4030CE00h	Reserved	Reserved
4030CE04h	Undefined	PC = [4030CE24h]
4030CE08h	SWI	PC = [4030CE28h]
4030CE0Ch	Pre-fetch abort	PC = [4030CE2Ch]
4030CE10h	Data abort	PC = [4030CE30h]
4030CE14h	Unused	PC = [4030CE34h]
4030CE18h	IRQ	PC = [4030CE38h]
4030CE1Ch	FIQ	PC = [4030CE3Ch]
4030CE20h	Reserved	20090h
4030CE24h	Undefined	20080h
4030CE28h	SWI	20084h
4030CE2Ch	Pre-fetch abort	Address of default pre-fetch abort handler ⁽¹⁾
4030CE30h	Data abort	Address of default data abort handler ⁽¹⁾
4030CE34h	Unused	20090h
4030CE38h	IRQ	Address of default IRQ handler
4030CE3Ch	FIQ	20098h

Figura 1: Tabela de Vetores de exceção de RAM retirado do manual de referência.

Para demonstrar como alguém pode carregar um endereço para uma função, devemos ir para o código em Assembly. Nosso manipulador de interrupção (Handler) precisará de pelo menos um pouco de assembly para capturar os registradores.

```

1 /* Startup Code */
2 _start:
3
4  /* Set V=0 in CP15 SCTRL register - for VBAR to point to vector */
5  mrc    p15, 0, r0, c1, c0, 0    // Read CP15 SCTRL Register
6  bic    r0, #(1 << 13)           // V = 0
7  mcr    p15, 0, r0, c1, c0, 0    // Write CP15 SCTRL Register

```

```
8
9  /* Set vector address in CP15 VBAR register */
10 ldr    r0, =_vector_table
11 mcr    p15, 0, r0, c12, c0, 0 //Set VBAR
12
13 /* init */
14 mrs r0, cpsr
15 bic r0, r0, #0x1F           // clear mode bits
16 orr r0, r0, #CPSR_SVC      // set SVC mode
17 orr r0, r0, #(CPSR_F)      // disable FIQ
18 and r0, r0, #~(CPSR_I)     // enable IRQ
19 msr cpsr, r0
20
21 /* IRQ Handler */
22 ldr r0, =_irq
23 ldr r1, =.irq_handler
24 str r1, [r0]
25
26 bl main
27
28 .loop:    b .loop
29
30
31 .irq_handler:
32 stmfd sp!, {r0-r12, lr}
33 MRS r11, spsr
34 bl ISR_Handler
35 dsb
36 msr spsr, r11
37 ldmfd sp!, {r0-r12, lr}
38 subs pc, lr, #4
```

O que esse código faz é carregar dois registradores separados com os valores contidos nas variáveis globais `_irq` e `.irq_handler`. Em seguida, ele armazena o valor de `r1` no endereço contido em `r0`. Com isso, podemos escrever a localização da função `.irq_handler` na tabela de abstração do vetor de interrupção. Sempre que um evento de IRQ ocorrer, o Beaglebone invocará nossa função Assembly.

Um manipulador de interrupção (handler) deve realizar algumas tarefas específicas imediatamente. A primeira coisa que ele deve fazer é armazenar todos os registradores atuais (r0–r12), bem como a instrução branch-back (lr). Isso será efetivamente um snapshot do que estava fazendo antes de ser interrompido. Onde o sistema armazena esses valores, exatamente? Bem, a resposta está no ponteiro da pilha. Precisamos inicializar o stack pointer para que, quando o sistema tentar tirar esses snapshot, tenha um local para armazenar os valores.

Fluxo Típico de Atendimento de Interrupção:

1. Periférico gera um evento e sinaliza ao INTC;
2. O INTC verifica a prioridade e, se a interrupção não estiver mascarada, sinaliza IRQ/FIQ ao núcleo ARM;
3. O processador salva o contexto e desvia para o vetor de exceção (0x00000018 para IRQ, 0x0000001C para FIQ, ou os equivalentes em modo high-vectors);

4. O handler lê o registrador SIR_IRQ/SIR_FIQ para identificar qual periférico gerou a interrupção;
5. Executa a ISR (Interrupt Service Routine) do periférico;
6. Ao final, escreve no registrador INTCPS_CONTROL para sinalizar o fim da interrupção (EOI – End of Interrupt), permitindo que o INTC libere a próxima interrupção pendente;
7. O processador restaura o contexto e retorna à execução normal.

O AM335x possui 128 interrupções periféricas, todas configuráveis como IRQ. Cada periférico do SoC (timers, UART, SPI, I2C, USB, Ethernet, ADC, GPIO, etc.) possui um número fixo de interrupção mapeado para o INTC. Por exemplo: Os timers de propósito geral (DMTimers) utilizam as linhas de interrupção 66, 67, 68, 69, 92, 93, 94 e 95 do AINTC. abaixo temos um exemplo de função para iniciar interrupção no AM335x:

```

1 void IntAINTCInit(void) {
2     /* Reset the ARM interrupt controller */
3     HWREG(SOC_AINTC_REGS + INTC_SYSCONFIG) = INTC_SYSCONFIG_SOFTRESET;
4
5     /* Wait for the reset to complete */
6     while ((HWREG(SOC_AINTC_REGS + INTC_SYSSTATUS) &
7             INTC_SYSSTATUS_RESETDONE) != INTC_SYSSTATUS_RESETDONE);
8
9     /* Enable any interrupt generation by setting priority threshold */
10    HWREG(SOC_AINTC_REGS + INTC_THRESHOLD) =
11        INTC_THRESHOLD_PRIORITYTHRESHOLD;
12 }

```

2 Configurando Interrupção para GPIO

Cada módulo GPIO possui uma linha de interrupção dedicada conectada ao MPU_INTC. As linhas de interrupção de cada banco no INTC são mostrados na Tabela 2.

Tabela 2: Registradores base para interrupção módulo GPIO.

Banco	Endereço Base	Linha INTC
GPIO0	0x44E07000	96
GPIO1	0x4804C000	98
GPIO2	0x481AC000	32
GPIO3	0x481AE000	62

Para programar a interrupção no GPIO precisamos definir qual grupo de interrupções vamos utilizar, no processador AM335x temos duas interrupções por módulo de GPIO (GPIOINTXA e GPIOINTXB - com X sendo o módulo GPIO), que permite rotear dois grupos de pinos GPIO para interrupções separadas, respectivamente ISRs separados que os atendem.

Agora precisamos definir qual registrador INTC_MIR_CLEARn configurar, e para isso, devemos descobrir o número da interrupção do grupo do módulo GPIO escolhido, como exemplo vamos escolher o GPIO módulo 1, e escolher o registrador de grupo de interrupção GPIO-INT1A, consultando a tabela “ARM Cortex-A8 Interrupts” na subseção 6.3, temos que o número da interrupção desse registrador é 98, então realizamos um deslocamento de 5 ($98 \gg 5$)

para descobrir qual valor de n do registrador `MIR_CLEARn`, que é 3, logo temos que configurar o registrador `INTC_MIR_CLEAR3` (0x4820_00E8), que tem base `INTC` (0x4820_0000) mais o offset (0xE8).

NOTE: Para programar o registro `INTC_MIRn`, os registros `INTC_MIR_SETn` e `INTC_MIR_CLEARn` são fornecidos para facilitar o mascaramento, mesmo que seja possível para compatibilidade com versões anteriores escrever diretamente no registro `INTC_MIRn`.

Agora precisamos saber qual bit será configurado para esse número de interrupção (98 = 0x62 em hexa), e para isso realizamos uma lógica (and) do número da interrupção com o valor (0x1F). Logo, para configurar a interrupção para os GPIOs do módulo 1 no grupo `GPIOINT1A` temos que setar o bit 2 (0x62&0x1F) do registrador `INTC_MIR_CLEAR3`, como mostrado na seguinte linha no código depois de configurar o clock do GPIO1:

```
1 /* Interrupt mask */
2 HWREG(INTC_MIR_CLEAR3) |= (1<<2); //98->bit 2 on register MIR_CLEAR3
```

Para gerar uma solicitação de interrupção para um evento definido (nível ou transição lógica) ocorrendo em um pino GPIO, os registradores de configuração GPIO devem ser programados da seguinte forma:

- As interrupções do GPIO devem ser habilitadas nos registros `GPIO_IRQSTATUS_SET_0` e/ou `GPIO_IRQSTATUS_SET_1`.
- Os eventos esperados no GPIO de entrada para acionar a solicitação de interrupção devem ser selecionados nos registradores `GPIO_LEVELDETECT0`, `GPIO_LEVELDETECT1`, `GPIO_RISINGDETECT` e `GPIO_FALLINGDETECT`.

Por exemplo, a geração de interrupção em ambas as bordas é configurada definindo como 1 nos registradores `GPIO_RISINGDETECT` e `GPIO_FALLINGDETECT` junto com a habilitação de interrupção para uma ou ambas as linhas de interrupção (`GPIO_IRQSTATUS_SET_n`).

Então para habilitar a interrupção no GPIO `gpio1_12` com detecção por borda de subida, precisamos configurar as seguintes linhas de códigos:

```
1 /* Setting interrupt GPIO pin. */
2 HWREG(GPIO1_IRQSTATUS_SET_0) |= 1<<12;
3
4 /* Enable interrupt generation on detection of a rising edge.*/
5 HWREG(GPIO1_RISINGDETECT) |= 1<<12;
```

NOTE: podemos habilitar a característica de **debouncing** na detecção do evento, somente setando 1 no bit específico do registrador `GPIO_DEBOUNCENABLE`.

2.1 Rotina de Serviço de Interrupção - ISR

Quando o evento de interrupção ocorrer (nesse caso o botão ser apertado), o processador invoca o tratador de interrupção, que no nosso sistema é definido por “`irq_handler`”, que por sua vez chamada a rotina de serviço de interrupção (ISR), representado pela função **ISR_Handler**

no código em C. quando a ISR é executada, primeiro se verifica qual tipo de interrupção (definido na tabela 6.1 da subseção 6.3 do manual técnico), para isso devemos fazer uma operação lógica (and) do registrador INTC_SIR_IRQ com 0x7F (em que os seis primeiros bits 0-6 definem o número da interrupção ativa). Com isso tomar alguma decisão e em seguida reconhecer a IRQ, como mostrado nas linhas de código abaixo:

```
1 void ISR_Handler() {
2     unsigned int intrNum;
3
4     intrNum = HWREG(SOC_AINTC_REGS + INTC_SIR_IRQ) & 0x7F;
5
6     switch (intrNum) {
7         case SYS_INT_GPIOINT1A:
8             gpioIsr();
9             break;
10        default:
11            break;
12    } /* ----- end switch ----- */
13
14    /* acknowledge the IRQ */
15    HWREG(SOC_AINTC_REGS + INTC_CONTROL) = 0x1;
16 }
```

Já na função “gpioIsr()” deve-se limpar o status dos flags de interrupção e também realizar alterações necessárias para ser utilizado em seu código, como mudar a flag de GPIO para alterar a sequência do piscar LEDs. Como visto na linha de código abaixo:

```
1 /* Clear the status of the interrupt flags */
2 HWREG(SOC_GPIO_1_REGS + GPIO_IRQSTATUS_0) = 0x1000;
3
4 flag_gpio = true;
```

3 Configurando Interrupção para Timer

O DMTimer é um timer de 32 bits que contém um contador livre crescente com capacidade de auto-reload no overflow. O contador pode ser lido e escrito em tempo real enquanto conta. O módulo inclui lógica de comparação para permitir um evento de interrupção quando o contador atinge um valor programável.

Para programar a interrupção do Timer precisamos definir qual grupo de Timer vamos utilizar e então habilitar sua interrupção, no processador AM335x temos uma interrupção por grupo de Timer, que permite rotear a interrupção separadamente, respectivamente ISRs separados que os atendem.

Agora precisamos definir qual registrador INTC_MIR_CLEARn configurar, e para isso, devemos descobrir o número da interrupção do grupo do Timer escolhido, como exemplo vamos escolher o DMTIMER7, consultando a tabela “ARM Cortex-A8 Interrupts” na subseção 6.3, temos que o número da interrupção desse registrador é 95, então realizamos um deslocamento de 5 ($95 \gg 5$) para descobrir qual valor de n do registrador MIR_CLEARn, que é 2, logo temos que configurar o registrador INTC_MIR_CLEAR2 (0x4820_00C8), que tem base INTC (0x4820_0000) mais o offset (0xC8).

NOTE: Para programar o registro `INTC_MIRn`, os registros `INTC_MIR_SETn` e `INTC_MIR_CLEARn` são fornecidos para facilitar o mascaramento, mesmo que seja possível para compatibilidade com versões anteriores escrever diretamente no registro `INTC_MIRn`.

3.1 Função Delay com Interrupção

```
1 unsigned int countVal = TIMER_OVERFLOW - (mSec * TIMER_1MS_COUNT);
2
3 /* Wait for previous write to complete */
4 DMTimerWaitForWrite(DMTIMER_WRITE_POST_TCRR, SOC_DMTIMER_7_REGS);
5
6 /* Load the register with the re-load value */
7 HWREG(SOC_DMTIMER_7_REGS + DMTIMER_TCRR) = countVal;
8
9 flag_timer = false;
10
11 /* Enable the DMTimer interrupts */
12 HWREG(SOC_DMTIMER_7_REGS + DMTIMER_IRQENABLE_SET) = 0x2;
13
14 /* Start the DMTimer */
15 DMTimerEnable(SOC_DMTIMER_7_REGS);
16
17 while(flag_timer == false) {}
18
19 /* Disable the DMTimer interrupts */
20 HWREG(SOC_DMTIMER_7_REGS + DMTIMER_IRQENABLE_CLR) = 0x2;
```

Quando habilitamos o TIMER com interrupção, então devemos carregar o registrador TCRR com o valor de tempo que deve ser contado. Com isso habilitamos a interrupção do timer (TIMER7) para que o hardware possa contar e no fim da contagem acionar a interrupção certa, como visto na rotina de interrupção apresentado na próxima subseção. Por fim desabilitamos a interrupção no fim da contagem, como vista no código acima.

3.2 Rotina de Serviço de Interrupção - ISR

Quando o evento de interrupção ocorrer (nesse caso o botão ser apertado), o processador invoca o tratador de interrupção, que no nosso sistema é definido por “irq_handler”, que por sua vez chamada a rotina de serviço de interrupção (ISR), representado pela função **ISR_Handler** no código em C. quando a ISR é executada, primeiro se verifica qual tipo de interrupção (definido na tabela 6.1 da subseção 6.3 do manual técnico), para isso devemos fazer uma operação lógica (and) do registrador `INTC_SIR_IRQ` com `0x7F` (em que os seis primeiros bits 0-6 definem o número da interrupção ativa). Com isso tomar alguma decisão e em seguida reconhecer a IRQ, como mostrado nas linhas de código abaixo:

```
1 void ISR_Handler() {
2     unsigned int intrNum;
3
4     intrNum = HWREG(SOC_AINTC_REGS + INTC_SIR_IRQ) & 0x7F;
```

```

5
6  switch (intrNum) {
7      case SYS_INT_TINT7:
8          DMtimerIsr();
9      break;
10     case SYS_INT_GPIOINT1A:
11         gpioIsr();
12     break;
13     default:
14     break;
15 } /* ----- end switch ----- */
16
17 /* acknowledge the IRQ */
18 HWREG(SOC_AINTC_REGS + INTC_CONTROL) = 0x1;
19 }

```

na função “DMtimerIsr” deve-se limpar o status dos flags de interrupção e também realizar alterações necessárias para ser utilizado em seu código, como visto na linha de código abaixo:

```

1 /* Clear the status of the interrupt flags */
2 HWREG(SOC_GPIO_1_REGS + GPIO_IRQSTATUS_0) = 0x1000;
3
4 flag_gpio = true;

```

3.3 Circuito Completo

Nessa prática você irá demonstrar o conhecimento adquirido na disciplina, durante as práticas realizadas, que envolve os procedimentos para configurar pino de GPIO (general purpose input/output) como entrada e saída, e no caso do pino de entrada utilizar o conhecimento de interrupção (nesse caso usando o pino 60). Por fim realizar a configuração do módulo de TIMER, para criar uma função Delay com valores de tempos passados pelo usuário, num menu.

3.4 Circuito Completo

Para conectar os botões, e os LEDs na BeagleBone, siga estes passos e consulte o diagrama para o circuito mostrado na Figura 2.

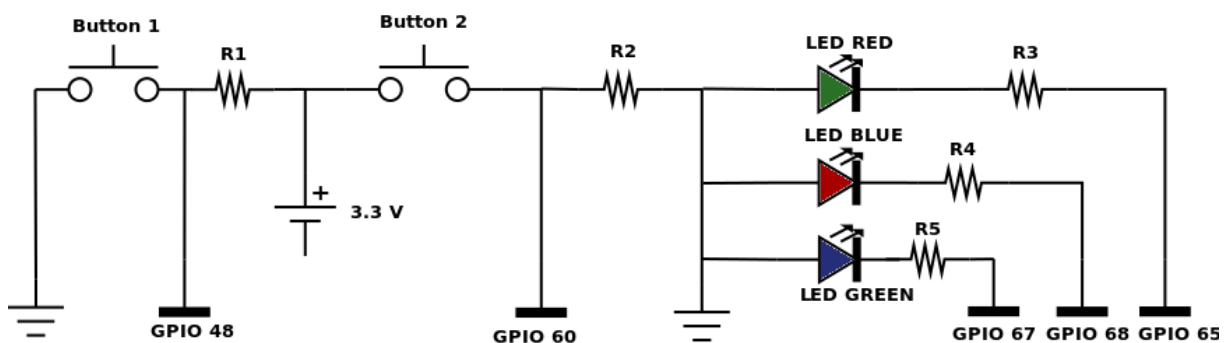


Figura 2: Valor do tempo no registrador TCRR.

Para conectar botões e LEDs para o BeagleBone, construa na protoboard um circuito resistivo acima. Em seguida faça os próximos passos para configurar o sistema completo:

1. **Configure o terra:** Conectar o pino GND do BeagleBone, por exemplo, pinos 1 e 2 em ambos os expansores - a faixa negativa da protoboard.
2. **Ligue o pino GPIO 60 para a protoboard:** Este GPIO 60 (gpio1_28) será utilizado como Button_2 (faça um pullDown) e terá a finalidade de alternar uma sequência qualquer de blink dos 3 LEDs, essa configuração deve ser realizada obrigatoriamente com interrupção.
3. **Conecte o GPIO_67, GPIO_68 e GPIO_65 para a protoboard:** Esses GPIOs serão utilizados para realizar uma sequência qualquer de blink definida pelo aluno.
4. **Ligue o pino GPIO_48 para a protoboard:** Este GPIO_48 (gpio1_16) será utilizado como Button_1 (faça um pullUp) e terá a finalidade de mostrar um menu para o usuário escolher qual o tempo em milissegundos da frequência do blink.

4 Atividades Práticas

pratica 1:

Baseado no código exemplo “gpio_int”, faça uma atualização do seu sistema para receber interrupção de botão.

pratica 2:

Utilizando o seu sistema de timer, faça um sistema que receba informação da uart para alterar em tempo real a frequência de oscilação do LED.